Firmware New Member Project

Florida Rocket Lab

Fall 2026

Introduction

This project is intended to teach you the fundamentals required to become a contributing member of Florida Rocket Lab's (FRL) Firmware system. This project will not teach you everything you need to know about the work we do, but it will give you a great starting point to build off of.

This project is meant to be difficult. However, if you are willing to put in time and effort, you will get a lot out of it. You will become a full-fledged member of FRL, and you will have a major boost to your future career endeavors. The topics you will learn are directly applicable in real-world industries, and you should be adding every skill learned and task finished to your resume.

This project has no prerequisites, but a basic understanding of electronics and programming will go a long way.

Project Structure

This new member project will last 6 weeks. For that duration you will be assigned a mentor (a current member on the firmware team), who will assist you in completing this project. They will be the first line of contact if you need help. In the first 4 weeks, you will be completing 3 programming assignments. To submit your assignment, you will meet with your mentor to have a discussion about the work you did that week. The meeting can be virtual or in person. You will be asked questions about your work and thought process and then graded based on your responses. Meetings must be scheduled by Wednesday night and they will run from Thursday to Friday at a time which you and your mentor will both agree on. You must submit your assignment to your mentor at least 30 minutes before the meeting. For the final deliverable, you will meet with your mentor halfway through the assignment and then have a final meeting the following week. Once you have successfully completed this meeting, you have finished the new member project and can now officially join the team.

Project Overview

1. Module I: C/C++ Fundamentals and Control Logic
   (a) Deliverable 1: Rocket Telemetry Logger (1 Week)
   (b) Deliverable 2: Rocket Telemetry Logger with Pointers and Functions (1 Week)
   (c) Deliverable 3: Rocket Telemetry Logger with a Cooperative Scheduler (1 Week)
2. Module II: STM32 HAL
   (a) Deliverable 4: I2C Sensor Driver (2 Weeks)

Project Timeline

- 9/28: Start Date
- 9/29: Flight Vehicles Orientation
- TBD: Firmware Orientation
- 10/7: Meeting Scheduled for Deliverable 1
- 10/8 - 10/9: Meetings for Deliverable 1
- 10/14: Meeting Scheduled for Deliverable 2
- 10/15 - 10/16: Meetings for Deliverable 2
- 10/21: Meeting Scheduled for Deliverable 3
- 10/22 - 10/23: Meetings for Deliverable 3
- 10/28: Meeting Scheduled for Deliverable 4
- 10/29 - 10/30: Meetings for Deliverable 4
- 11/4: Final Meeting Scheduled for Deliverable 4
- 11/5-11/6: Final Meeting Scheduled for Deliverable 4

1 Module I: C/C++ Fundamentals and Control Logic

1.1 Step 1: Programming in C/C++

1.1.1 Background

Avionics refers to the electronic systems used in aviation, rocketry, satellites, and other flying machines. These systems range from simple flight controllers with radios to the complex life-support and guidance systems in spacecraft.

Most avionics share the same core functions: sensing data (altitude, velocity, orientation), processing it through onboard software, acting on it by controlling hardware (engines, motors, valves), and sending telemetry back to the ground. For our purposes, avionics means building a flight computer that reads sensors, runs control logic, and communicates with the rest of the rocket.

To do this, two things are required: hardware (designing and wiring components into a reliable system) and firmware (the code that drives the hardware to behave predictably). Embedded systems are specialized computers designed for dedicated tasks. Unlike general-purpose PCs, they tightly integrate hardware and software to monitor inputs, process data in real time, and control outputs.

The main tool we'll use to control them is the C programming language, which provides low level control without high level abstractions such as classes, which are often unnecessary for embedded programming. It's efficient, reliable, and perfect for avionics. Now, lettuce begin.

The slides provided below are from the COP3504 course at UF, which teach C++. Although these slides do not teach C explicitly, they go over fundamentals which apply to both languages. Most of the concepts you learn will transfer over to C with little to no changes. You by no means have to go through ALL of these, but it's highly encouraged.

However, if you already know how to code in C or C++, reading these slides is not required and you can move straight into the two programming assignments in this section. The link to access these are down below, just make sure you're signed into your UFL google account.

NOTE: COP3504 is the advanced programming class at UF, which unites programming 1 and 2 into a single semester course. As a result it's rather intensive and comprehensive. The goal is NOT for you to be a C/C++ programming expert, instead it's meant for you to have a solid grasp of the basics before we start using it to program microcontrollers. For each section use this document as your guide to what you should focus on and retain. You're welcome to dive deeper into the full content, but keep in mind that much of it goes beyond what's necessary for our use case.

- 2: How computers work (part A)
  - This section of the slides introduces how computers actually work at the most basic level. It explains on the flow from instructions to the processor to the hardware, and goes into details like offsets, transistors, and lower-level circuitry. Those details are interesting, but not important for our purposes. What you should focus on is building an intuitive grasp of what's going on: a computer takes in instructions (programs), stores and moves them around in memory, and executes them step by step in hardware.
  - KEY CONCEPT: A computer is a machine that takes instructions (written by us), turns them into binary signals and executes them step by step in hardware. You don't need the hardware details, just know that this is the foundation of everything else in programming.
- 3: How computers work (part B)
  - This section introduces how computers represent and manipulate information at their most fundamental level. You'll learn the difference between bits (the smallest unit of information, a 1 or 0) and bytes (groups of 8 bits that form usable data). The slides then connect this to how programs are written, showing that while humans write instructions in programming languages like C++, the computer itself only understands binary machine code. Translators like compilers, assemblers, and interpreters bridge the gap. Turning human friendly code into precise electrical instructions a CPU can execute.
  - Understand the idea that everything reduces to 1s and 0s at the hardware level.

  - Recognize why humans use higher-level languages (easier to write, read, and maintain).
  - Grasp the role of compilers and interpreters as "middle-men" between your code and the CPU.
  - Don't worry about memorizing how instructions are encoded. Just know that this translation process is what allows your Arduino code to "become real."
  - KEY CONCEPT: Computers only understand machine code (1s and 0s), but humans write in higher level languages. Compilers, assemblers, and interpreters act as translators, turning human friendly code into instructions that a CPU's circuits can execute.

- 5: Basic compiler information
  - Basic compiler information, how the preprocessor works, how include works, <iostream> and similar libraries.
  - You don't need to memorize how compilers work at this stage. Just know that the IDE is handling all the background steps (preprocessing, compiling, linking) for you whenever you hit the "Run" button. Your focus should be on writing and understanding the C++ code itself, not on compiler command line details.
  - KEY CONCEPT: To use cout/cin, you need to include the iostream header. Normally, you would compile your code with g++ in the terminal, which turns your human-readable .cpp file into an executable program the computer can run. However, for this project we will be using CLion as our development environment. CLion handles the compiling automatically in the background, so you won't be typing g++ commands yourself. Instead, you just press "Run" in CLion, and it will take care of compiling and executing the program for you.

- 6: Primitive Data Types
  - Primitive data types are the building blocks of every program. This section covers booleans, integers (signed vs. unsigned, different sizes, fixed-width), and floating-point numbers. It also introduces binary representation, two's complement, overflow/underflow, and the concept of bounds. Floats are explained in terms of scientific notation and their trade-offs with precision. You don't need to memorize IEEE-754 specifics right now, but you should understand that data types come in different sizes and each is suited for different purposes.
  - Make sure you understand the difference between integers and floats, what signed vs. unsigned means, and the idea that computers store numbers in binary. Know that numbers have limits (overflow and underflow) and that floats can represent very large or small values but lose precision. These concepts will come up constantly when programming in C++.
  - KEY CONCEPT: C++ primitive types are bool, int (various sizes), and floats (float/double/long double). Integers use binary two's complement, can overflow/underflow, and vary by platform. Floats handle huge/small numbers with scientific notation but have precision limits.

- 7: Type Casting and Type Conversions
  - Type casting is the process of converting a variable from one data type to another. In C++, this is an essential tool because different operations and functions may require different types. The slides cover implicit conversions (done automatically by the compiler), explicit conversions (done manually by the programmer), and the differences between safe and unsafe casts.
  - For our purposes, don't worry about all the technical edge cases in the slides (like when the compiler silently promotes certain types). Instead, focus on understanding why conversions happen, and the difference between widening (broadening, usually safe) vs. narrowing (can lose data, risky).

  - KEY CONCEPT: Conversions can either preserve information (broadening) or lose information (narrowing). Use static cast instead of old C-style casts to make conversions explicit and safe.

- 8: Literals and Suffixes
  - Literals are the fixed values you actually type into your code, like 42, 3.14, or 'a'. Each literal has a default type depending on what it looks like (for example, 42 is by default an int, while 3.14 is a double). C++ lets you modify or clarify what type a literal should have by adding a 19 suffix. For instance, writing 42u makes it an unsigned int, and 1.0f makes it a float. This is particularly important when dealing with large numbers or when you need to ensure the literal matches the type you want to work with.
  - KEY CONCEPT: Literals are fixed values in code, each with a default type. You can change their type explicitly by adding suffixes (like 42u for unsigned int, 1.0f for float).

- 9: Primitive Data Types (Concept Check)
  - This is not something you need to memorize perfectly, but it's important to honestly test yourself and see which concepts stick and which you should review again.
  - Don't worry about focus understand, if you can do the following you're good:
    * The difference between integer and floating-point types.
    * What overflow and underflow mean for integers.
    * How signed vs unsigned integers affect the range of values.
    * That floating-point numbers (float, double) trade precision for range.

- 10: Variable Declaration and the Stack
  - This is one of the most important modules yet. You will be declaring variables and working with the stack in every program you write, so it's critical to understand how variables live in memory and how scope works. This module covers static typing, declaration syntax, uninitialized variables, variable lifetime, scope, and how data is stored on the stack versus the heap. Don't stress about all the tiny implementation details, but do make sure you can explain the difference between stack and heap memory and why scope matters.
  - Variables must be declared before use, uninitialized variables may contain garbage values.
  - The stack stores local variables and manages scope automatically (created when entered, destroyed when exited).
  - The heap used for dynamic memory allocation (we'll touch on this more later).
  - Scope defines where a variable can be accessed, misuse can lead to shadowing or dangling references (not good as you can easily create memory leaks).
  - KEY CONCEPT: C and C++ are statically typed, every variable has a type that must be known at compile time.

- 11: Variable Declaration and the Stack (Concept Check)
  - This section is a short concept check covering variable declaration, scope, the stack, and the heap. You don't need to memorize every detail, but you should be able to match terms with their definitions and understand how variables behave in memory. The goal is to make sure the fundamentals are solid before moving on, since these concepts directly affect how your programs will run on an Arduino.
  - KEY CONCEPT: Verify stack/heap/scope fundamentals before touching hardware. Variables live on the stack and disappear when out of scope, while the heap is used for dynamically allocated memory. Misunderstanding these ideas can cause bugs like memory leaks, dangling references, or crashes. All of which are serious pains in the ass to figure out what you did wrong and how to fix it.

- 12: Common C++ Operations
  - Operations in C++ include basic arithmetic, comparisons, boolean logic, and bitwise manipulation. Many of these should feel familiar if you've programmed in another language, but C++ has some unique details. In particular, the difference between integer division and floating-point division is crucial, as well as how booleans interact with integers. Also pay attention to short-circuiting in logical operators (&&, ||), which stops evaluation early if the result is already determined. These operations form the backbone of nearly every program you'll write. 20
  - KEY CONCEPT: C++ supports standard math (+, -, *, /, %), comparison (==, !=, <, >, <=, >=), boolean (&&, ||, !), and bitwise (&, |, ^, <<, >>) operations. Integer division truncates results while floating-point division keeps decimals. Logical operators can short-circuit, which improves efficiency but may skip evaluating later expressions.
- 13: Control Flow
  - This is one of the most important concepts for beginners. Control flow is what gives a program "decision-making" and "repetition." Without it, code would just run top-to-bottom like a static list of instructions. These slides cover the basics of conditionals and loops: if, if-else, switch, the ternary operator (?:), and the three major kinds of loops (while, do-while, and for).
  - Focus here on understanding the logic behind how programs make choices and repeat tasks. Don't stress too much about memorizing every detail, practice writing simple examples will make this stick.
  - KEY CONCEPT: Control flow decides what code runs and how often. Conditionals (if, else, switch, ?:) choose between paths, and loops (while, do-while, for) repeat code until a condition is met.
- 14: Basic C++ Syntax (Concept Check)
  - This section is a short progress check that challenges you on the fundamentals of C++ syntax. Don't worry if it feels tricky, the point is not to memorize everything. But to ensure you've internalized the basics before moving on. You can keep the slides open while working, but try to answer from memory first. That way, you'll know where your weak spots are.
- 15: Arrays and Vectors
  - Arrays and vectors let us store and work with collections of values instead of just single variables. They are absolutely essential because they form the foundation for organizing and managing data in almost every program you'll write (big ones anyways). If you've never worked with arrays before, take your time here to understand them since it'll make the rest of your tasks much easier.
  - KEY CONCEPT: Arrays are the simplest way to store multiple values, but they're fixed size and unsafe if misused. Vectors are the safer, flexible alternative in modern C++ that automatically manage memory and provide helpful functions.
- 16: Strings and Streams
  - Strings are fundamental in programming, but in C++ they're tricky. C-style strings rely on a special ‘‘ terminator, which can easily cause bugs if forgotten. Luckily, std::string makes working with text much safer and easier. Streams, on the other hand, are essential to almost everything we do. They handle input and output. Every time you type something in, print data out, or move information to/from files, you're using streams. Since avionics deals with live telemetry, mastering streams is critical for reliably sending, receiving, and formatting data.

– KEY CONCEPT: Strings store text. C-style strings are character arrays ending with ‘‘, but std::string is the safer modern option. Streams (cin, cout, file streams, string streams) provide a unified way to move data in and out of programs. Understanding them is crucial for telemetry and communication in our flight computer.

Make sure you enjoy that cookie since you've deserved it. If you have any questions what's so ever please do not hesitate to ask and to look over the material again. C++ is inherently a difficult language to learn and work with so don't feel frustrated if the concepts do not work the first time around. Review and persistence matter more than memorization!!!

1.1.2 Tasks

1. Before moving on, we need to make sure your development environment is ready. CLion is the IDE (Integrated Development Environment) we will be using to write and run all of our C code. It takes care of the compiler setup (normally you would use g++ in the terminal), project organization, and running/debugging for you.
   - Follow along with this YouTube tutorial: Getting Started with CLion (Setup and First Project).
   - Install CLion (use your ufl account to get the free year of service) and configure it with the default settings. Make sure it detects your compiler toolchain automatically.
   - Create a new C project in CLion, add a simple main.c, and run a "Hello World" program.
   - Verify that the output prints correctly in the run window at the bottom of CLion
2. Practice implementing any of the examples found in the slides or elsewhere until you feel comfortable with the basics of coding in C/C++.

1.1.3 References

- Referenced slides - Make sure you're signed in with your UFL account.

1.2 Step 2: Rocket Telemetry Logger

1.2.1 Background

You are going to write a tiny C program that simulates vertical "launch" using very simple step-by-step updates. At each step, update altitude and velocity, burn a little fuel, save the altitude to a list, and print a short flight log at the end.

You should focus on the following:

- *Primitive types & literals:* **double** for altitude/velocity/time, **int** for step counters.
- *Variable declaration & scope:* keep variables in the smallest scope that makes sense.
- *Control flow:* a loop with simple **if/else** checks for stop conditions.
- *Arrays:* store altitude history in an array with a preset max size.
- *Streams*: use **scanf()/printf()** for clean I/O.

We are going to use a simplified model with only gravity and a constant upward push while fuel lasts. No drag, no changing mass.

$$a = \begin{cases} a_{thrust} - g & \text{if fuel} > 0 \text{ (thrusting)} \\ -g & \text{if fuel} = 0 \text{ (coasting/falling)} \end{cases}$$

a : Net acceleration
a_{thrust} : Upwards acceleration due to the engine
g : Downwards acceleration due to gravity (use 9.81)

1.2.2 Tasks

The user will provide the following inputs:

- Starting altitude z_0 (meters), starting velocity v_0 (m/s).
- Initial fuel (in "steps"; e.g., how many time-steps of thrust you can provide).
- Engine acceleration when thrusting, a_{thrust} (m/s^2) (a single constant number).
- Time step Δt (seconds), e.g. $\Delta t = 0.1$.
- Maximum number of steps (safety cap).

You will then implement a loop which updates in the following order via:

$$v \leftarrow v + a \cdot \Delta t$$
$$z \leftarrow z + v \cdot \Delta t$$

Then:

- If you were thrusting this step, decrease fuel by 1 (or whatever you prefer)
- Add the new z (altitude) into your history array
- End the simulation on touchdown if z $\leq$ 0 and v $\leq$ 0 after at least one step, or when the step counter reaches max_steps

After the loop you will print the following as a flight log:

- total simulated time and steps
- max altitude reached

- final altitude z and velocity v
- remaining fuel
- altitude history (all values or every Nth value for readability)

You will write this all in a .c file which must build and compile in CLion. Take a screenshot of the model working and write 2-3 sentences explaining your chosen inputs. Put this in a README.txt for your project.

Example I/O (sketch):

```
Enter z0 (m): 0
Enter v0 (m/s): 0
Enter initial fuel (steps): 50
Enter engine accel a_thrust (m/s^2): 15
Enter dt (s): 0.1
Enter max steps: 300

--- Rocket Telemetry ---
Steps: 124 Total time: 12.4 s
Max altitude: 103.0 m
Final altitude: 0.0 m
Final velocity: -45.1 m/s
Fuel remaining: 0
Altitude history (every 10 steps): 0.00, 2.85, 10.90 ...
```

Hints:

- Pick a small Δt (e.g. 0.1) to keep numbers smooth.
- If $a_{thrust} < g$, you may never lift off. that's okay, it's good testing.
- Clamp altitude to ≥ 0 when printing if you want to avoid negative values on "landing".
- The size of the array should be set to the max amount of steps as decided by the user.

To submit the assignment:

-

1.3 Step 3: Pointers and References

1.3.1 Background

Pointers and references are critical in C/C++ because the language gives you direct control over memory management. Unlike higher level languages that abstract this away, C/C++ requires you to understand how data is stored and accessed. This makes your programs more efficient and powerful, but it also means you have to be more careful.

At first, pointers and references can feel confusing and even intimidating. However, once mastered, they become one of the most valuable tools in your programming toolkit. They are essential to the foundation of embedded systems and controlling physical hardware would be impossible without them. Again, a reading guide for the slides is provided below.

- 2: What's the Point(er)
  - This section introduces pointers, one of the most important but also most confusing concepts in C++. A pointer doesn't hold the actual value of a variable. Instead, it stores the memory address where that value lives. This gives you direct control over memory and makes your programs faster and more flexible. However, it also comes with risks if used incorrectly (like dereferencing a nullptr). For our purposes, don't worry about transistors, offsets, or hardware-level details. Just focus on understanding what an address is, how to declare a pointer, and how to use the dereference operator (*) and address-of operator ().
  - KEY CONCEPT: A pointer holds the address of data, not the data itself. This makes programs more powerful and memory-efficient but requires careful handling. Understanding how memory is laid out (stack, heap, data, text) will help you grasp why pointers are both so useful and so dangerous.
- 3: The Plain-Old-Data Struct
  - Structs are an essential tool because they let us organize related pieces of data into one reusable unit. Instead of juggling separate variables, a struct groups them together into a single object that can be passed around and reused in larger, more complex programs. This makes your code more structured, readable, and scalable.
  - KEY CONCEPT: A struct is just a container that groups variables into one object, making it easier to represent real-world entities. They're simple, flexible, and the foundation for more advanced constructs like classes.
- 4: A Couple C++ Pointers
  - This section introduces some of the most important (and trickiest) features of C++: pointers. A pointer is a variable that stores the memory address of another variable instead of its value. You'll see how the (address-of) and * (dereference) operators work, and how pointers connect to arrays, vectors, and strings. You'll also learn about contiguous memory and why pointers allow you to "walk" through it with pointer arithmetic.
  - Focus on understanding what an address is, how to declare and use pointers, and why they matter for efficiency. Don't worry about advanced pointer tricks here, just get comfortable with the idea that pointers give you direct control over where your data lives in memory.
  - KEY CONCEPT: A pointer doesn't hold a value, it holds an address where a value lives. This lets you reference, share, and work with data without making unnecessary copies. Arrays and pointers are closely related in C++, which is why array indexing and pointer arithmetic often look the same.
- 5: Array Decay and Pointer Magic

- This section explains one of the quirks of C++: arrays often "decay" into pointers. That means if you pass an array to a function, what really gets passed is just the memory address of its first element. This is why arrays and pointers feel so similar in C++. It also explains why pointer arithmetic (moving forward/backward in memory) and array indexing work the same way.
- You don't need to get bogged down in every detail here. The important part is understanding that arrays and pointers are deeply connected in C++ and that this gives you both power and risk. The compiler won't stop you from stepping outside array bounds, so responsibility falls on you to know what your program is doing and how it's accessing memory.
- KEY CONCEPT: Arrays automatically decay into pointers to their first element, making array indexing and pointer arithmetic two sides of the same coin. This is powerful but dangerous, misuse can cause memory errors.

1.3.2 Optional Tasks

1. Practice implementing any of the examples found in the slides or elsewhere until you feel comfortable with the basics of memory management in C++.

1.3.3 References

- Referenced slides - Make sure you're signed in with your UFL account.

1.4 Step 4: Telemetry with Pointers and Structs

1.4.1 Background

You are going to reuse the tiny rocket simulation from earlier (same super-simple physics). This time, you will "gather" data from two sensors: an accelerometer, which measures acceleration in the z axis, and a fuel sensor, which measures how much fuel is left in the tank. You will store the sensor data in a struct. Then, you will apply an algorithm to this struct to calculate the velocity and altitude of the rocket. These values will then be stored into another struct.

You will be focusing on the following concepts:

- *Header Files:* You will create a main.h file to define data types and functions
- *Structs:* You will use structs to represent complex data types
- *Enums:* You will use enums to represent states for a state machine
- *Functions:* You will use functions to encapsulate certain tasks
- *Pointers:* You will use pointers to pass data around into functions instead of making copies
- *Macros:* You will use macros to define necessary constants
- *Static Variables:* You will use static variables to define data in functions that should not change from call to call
- *Const Parameters:* You will be using the const keyword in functions when the argument is not modified

1.4.2 Tasks

1. Use macros to define constants that were previously user defined in step 2, such as initial altitude, velocity, fuel, thrust, and the time step.
2. Define the following struct data types in your header file:

```
typedef struct {
    double acceleration;
    uint16_t fuel;
    float timestamp;
} sensor_sample_s;

typedef struct {
    sensor_sample_s sensor;
    double velocity;
    double altitude;
} telemetry_packet_s;
```

3. Define the following functions in your header file:

```
void update_sensor_sample(sensor_sample_s *sensor);
void update_telemetry_state(const sensor_sample_s *sensor, telemetry_packet_s *telemetry);
void run_rocket_state(const telemetry_packet_s *telemetry);
```

The first function will take a sensor sample struct pointer and determine what the next acceleration and fuel remaining for the rocket will be. The second function will take a sensor sample and calculate velocity and altitude and put it into telemetry_packet_s. The third fucntion will take the current telemetry packet and determine the current state of the rocket. The state of the rocket should be a static global variable that gets modified inside of this function. (Hint: Use a switch case to define your state machine)

4. Define the following enum in your header file:

```
typedef enum {
    FLIGHT_ASCENT,
    FLIGHT_COAST,
    FLIGHT_DESCENT,
    FLIGHT_LANDED
} rocket_state;
```

5. Use the following loop in your main function:

```
int main(void) {
// Variable Definitions

    while (state != FLIGHT_LANDED) {
        // Run your loop in here
    }
}
```

Inside this loop you will put the functions you have written. Make sure to print something every loop so that you can make sure the calculations are correct.

6. Create a readme.txt file where you write 2-3 sentences describing your thought process for each function and for the main loop.

Note: Make sure you understand what all of the code snippets given to you are actually doing. They are provided so that you don't have to write excessive boilerplate and so you can focus on the underlying logic in the code.

1.4.3 References

Switch case state machines